

Sports Stream Backend Platform

Scalable live sports streaming infrastructure built on Go microservices & Google Cloud Platform

PROJECT NAME	sports-stream-backend
LANGUAGE / RUNTIME	Go 1.25 — compiled to static binaries
CLOUD PLATFORM	Google Cloud Platform (GCP) — Cloud Run / Kubernetes
DATABASE	Cloud Firestore (NoSQL, real-time)
AUTHENTICATION	Firebase Auth — JWT / Google Sign-In
CLIENT APP	Android (ExoPlayer HLS streaming)
ARCHITECTURE STYLE	Microservices — Gateway + 5 independent services
DEPLOYMENT	Docker container — single image, Cloud Run / K8s

"A production-grade live sports streaming platform — from RTMP ingest to HLS delivery, real-time viewer tracking, push notifications, and a full admin control panel."

Table of Contents

1 Project Overview	3
1.1 What is Sports Stream?	3
1.2 Key Numbers at a Glance	3
1.3 Technology Stack	3
2 System Architecture	4
2.1 Architecture Diagram	4
2.2 Service Responsibilities	4
2.3 Event-Driven Flow	5
3 Microservices Deep Dive	5
3.1 API Gateway (port 8080)	5
3.2 User Service (port 8081)	6
3.3 Stream Service (port 8082)	6
3.4 Notification Service (port 8083)	7
3.5 Admin Service (port 8084)	7
3.6 Analytics Service (port 8085)	8
4 API Reference	9
5 Data Model	10
6 Video Pipeline	11
7 Security & Authentication	11
8 Infrastructure & Deployment	12
9 Observability & Health	12
10 Future Roadmap (AWS Migration)	13

1 Project Overview What is Sports Stream?

1.1 What is Sports Stream?

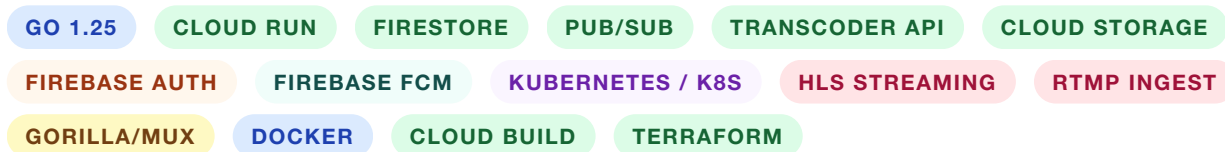
Sports Stream is a **live sports streaming backend platform** that powers a mobile Android application. It allows sports broadcasters to stream live matches via RTMP, which the platform transcodes into HLS (HTTP Live Streaming) format for delivery to viewers. The system handles everything from user authentication, real-time viewer tracking, push notifications, analytics collection, to full admin management.

The project was built for the **CCC'26 competition** and demonstrates a production-ready microservices architecture deployed on Google Cloud Platform.

1.2 Key Numbers at a Glance



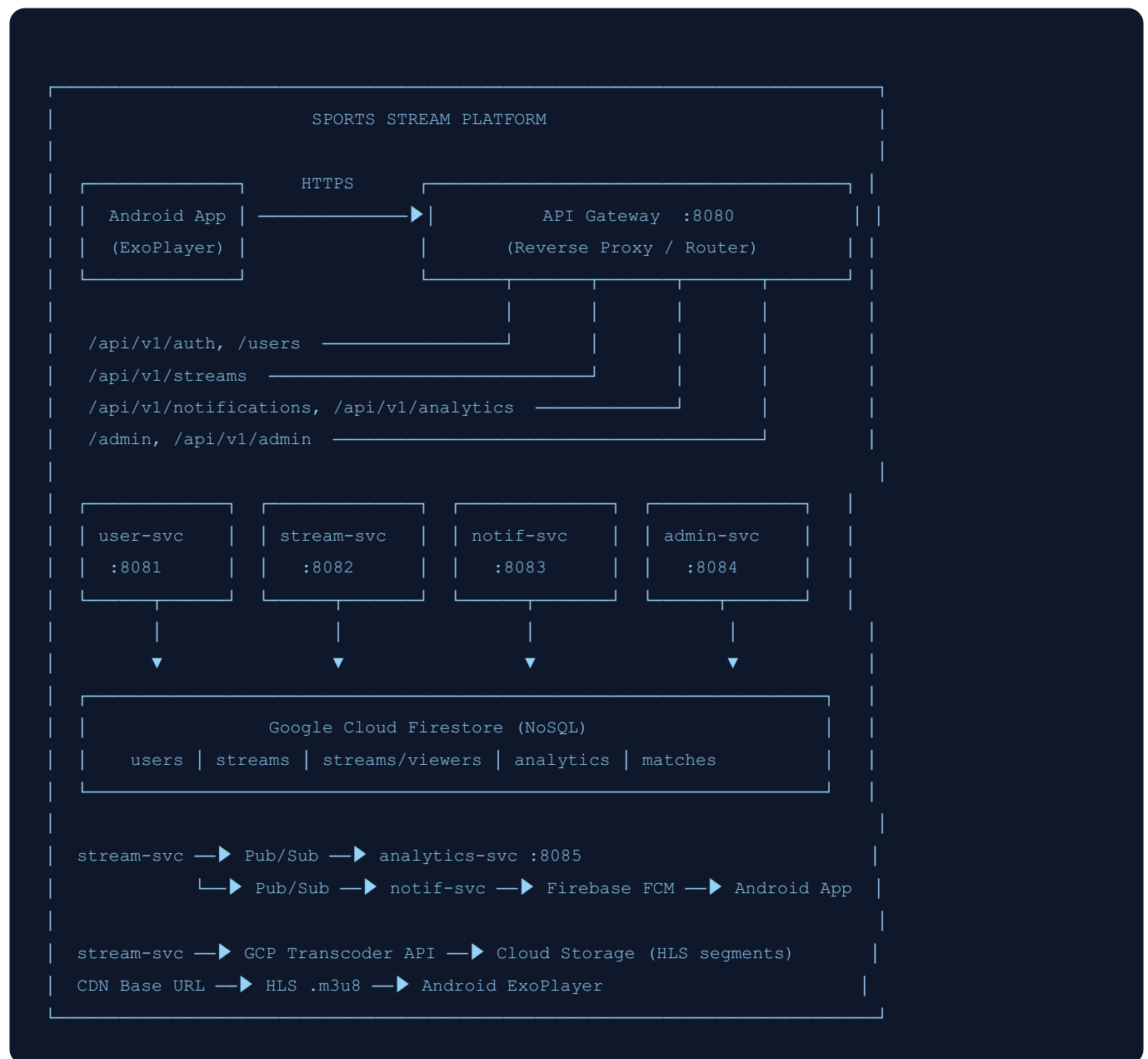
1.3 Technology Stack



Architecture Philosophy: Each service is independently compiled to a static binary, packaged together in a single Docker image, and started by a `start.sh` supervisor script. This "monorepo, multi-binary" approach gives microservice isolation while simplifying Cloud Run deployment to a single container.

2 System Architecture

2.1 Architecture Diagram



2.2 Service Responsibilities

API Gateway

Port 8080 — Public entry point

Routes all incoming requests to the appropriate microservice using prefix-based path matching. Injects request IDs, serves the landing page and health dashboard. The only service exposed externally.

User Service

Port 8081 — Identity & Profiles

Handles Firebase JWT verification on login, upserts user profiles in Firestore, serves profile read/update endpoints. New users are assigned the `viewer` role by default.

Stream Service

Port 8082 — Live stream management

Creates/lists streams, manages viewer join/leave using Firestore transactions with a subcollection for idempotent presence tracking. Triggers GCP Transcoder API for RTMP→HLS. Publishes events to Pub/Sub.

Notification Service

Port 8083 — Push notifications

Subscribes to `stream_events` Pub/Sub topic. Sends Firebase FCM push notifications to the `sports_live` and `match_reminders` topics when streams start, end, or matches approach.

Admin Service

Port 8084 — Operations control

Full CRUD for users, streams, matches, and analytics. Web-based admin panel with HMAC session cookies. Real-time analytics via Server-Sent Events (SSE). Serves the embedded SPA admin UI.

Analytics Service

Port 8085 — Viewer metrics

Subscribes to `viewer_events` Pub/Sub. Tracks `currentViewers`, `peakViewers`, `totalJoins` per stream using idempotent Firestore transactions with SHA-256 event fingerprinting to prevent duplicate processing.

2.3 Event-Driven Flow

The platform uses **Google Cloud Pub/Sub** for asynchronous communication between services:

stream_events topic

Published by stream-service when a stream starts or ends.

Subscribers: notification-service (sends FCM push), analytics-service (initializes/resets analytics doc)

viewer_events topic

Published by stream-service on every viewer join or leave action.

Subscribers: analytics-service (increments/decrements `currentViewers`, `peakViewers`, `totalJoins`)

3 Microservices Deep Dive

3.1 API Gateway (port 8080)

The gateway is a **pure HTTP reverse proxy** built with Go's standard `net/http/httputil` package. It has no authentication logic — all auth happens in downstream services.

PATH PATTERN	FORWARDS TO	NOTES
<code>/</code>	Landing page (inline HTML)	Shows live matches from <code>/api/v1/streams</code>
<code>/health</code>	Internal health check	Probes all 5 services, renders HTML dashboard
<code>/api/v1/auth/*</code>	user-service :8081	Token verification
<code>/api/v1/users/*</code>	user-service :8081	Profile management
<code>/api/v1/streams/*</code>	stream-service :8082	Stream CRUD & viewer tracking
<code>/api/v1/notifications/*</code>	notification-service :8083	Test notification trigger
<code>/api/v1/analytics/*</code>	analytics-service :8085	Viewer statistics
<code>/admin/*</code> / <code>/api/v1/admin/*</code>	admin-service :8084	Admin panel & APIs

3.2 User Service (port 8081)

Handles the complete **identity lifecycle** for the Android app users.

- **POST `/api/v1/auth/verify`** — Accepts a Firebase ID token (from Android Google Sign-In), verifies it with Firebase Admin SDK, then upserts the user profile in Firestore.
- **GET `/api/v1/users/me`** — Returns the authenticated user's full profile including role.
- **PATCH `/api/v1/users/me`** — Updates `displayName`, `favTeams`, and `fcmToken`.

User Roles: `viewer` (default), `broadcaster` (can create streams), `admin` (full access).

Role preservation: When an existing user logs in again, the service only updates email, `displayName`, and `photoUrl` — it never overwrites the role field. Promotions are done by admins via the admin panel or Firebase Console.

3.3 Stream Service (port 8082)

The core service managing **live stream lifecycle and viewer presence**.

- **GET `/api/v1/streams`** — Lists all `status: live` streams (max 50), merges `currentViewers` from analytics.
- **POST `/api/v1/streams`** — Creates a stream (broadcaster/admin only), triggers GCP Transcoder async.

- **POST /api/v1/streams/{id}/join** — Atomically increments viewerCount using a Firestore transaction. Idempotent — joining twice returns the current count without double-counting.
- **POST /api/v1/streams/{id}/leave** — Atomically decrements viewerCount, floored at 0. Safe across all Cloud Run instances.
- **GET /api/v1/streams/{id}/manifest** — Returns the HLS manifest URL for ExoPlayer.

Viewer Idempotency Pattern: A Firestore subcollection `streams/{id}/viewers/{uid}` stores presence docs. Join/leave transactions first check this subcollection — if a user is already marked present, the viewerCount is not incremented again. This eliminates double-counting across network retries and multiple Cloud Run instances.

3.4 Notification Service (port 8083)

An **event-driven FCM push notification dispatcher**. It runs both an HTTP server (for health checks and test triggers) and a Pub/Sub subscriber goroutine simultaneously.

- Subscribes to `stream_events` Pub/Sub topic.
- `stream_started` → sends "Match is LIVE!" to `sports_live` FCM topic.
- `stream_ended` → sends "Match Ended" to `sports_live` FCM topic.
- `match_reminder` → sends "Match starting soon!" to `match_reminders` FCM topic.
- On Pub/Sub errors, the subscriber retries with a 5-second backoff — the HTTP server stays up.

POST /api/v1/notifications/test — Allows testing notifications without Pub/Sub (direct API call).

3.5 Admin Service (port 8084)

A full **operations control center** serving both a REST JSON API and a single-page web application built into the binary itself (no external CDN dependencies).

Admin Panel Features

- Dashboard with live stats (users, streams, viewers)
- User management — list, create, update role, delete
- Stream management — list, create, end, reset viewers
- Match schedule management (CRUD)
- Analytics leaderboard with viewer metrics
- Real-time analytics via SSE (no polling)
- Send push notifications directly
- System health check for all services

Authentication

The admin panel supports two auth methods:

- 1. Session cookie (web UI):** HMAC-SHA256 signed token with username/password login. 12-hour expiry. HttpOnly, SameSite=Lax.
- 2. Firebase Bearer token (API):** Checks the caller has `role: admin` in Firestore.

3.6 Analytics Service (port 8085)

Processes viewer and stream events from Pub/Sub to maintain **real-time analytics aggregates** in Firestore.

Metrics Tracked per Stream

- **currentViewers** — live count, incremented/decremented per join/leave event
- **peakViewers** — maximum concurrent viewers ever recorded
- **totalJoins** — cumulative join count
- **viewerHistory** — subcollection with per-user join/leave timestamps and watch count

Idempotency Mechanism

Each event has an `eventId`. A `processedEvents` subcollection stores processed event IDs. All counter updates run inside Firestore transactions that first check if the event was already processed.

If no eventId is provided, one is generated as a SHA-256 fingerprint of

```
eventType + streamId + uid + timestamp
```


4 API Reference

Authentication Endpoints

METHOD	PATH	AUTH	DESCRIPTION
POST	/api/v1/auth/verify	None	Verify Firebase ID token. Body: <code>{"idToken": "..."} </code> . Returns full user profile with role.
GET	/api/v1/users/me	Bearer	Get authenticated user profile (uid, email, displayName, favTeams, role)
PATCH	/api/v1/users/me	Bearer	Update displayName, favTeams, or fcmToken (role cannot be changed by user)

Stream Endpoints

METHOD	PATH	AUTH	DESCRIPTION
GET	/api/v1/streams	None	List all live streams. Returns array with viewerCount + currentViewers (merged from analytics).
POST	/api/v1/streams	Bearer (broadcaster/admin)	Create a new stream. Body: <code>{"title":"...", "rtmpUrl":"..."}</code> . Triggers transcoder.
GET	/api/v1/streams/{id}	None	Get single stream details including merged currentViewers.
POST	/api/v1/streams/{id}/join	Bearer	Join a stream. Idempotent — returns current viewerCount. Returns 410 Gone if stream ended.
POST	/api/v1/streams/{id}/leave	Bearer	Leave a stream. Idempotent — no effect if not currently joined. Count never goes below 0.
GET	/api/v1/streams/{id}/manifest	Bearer	Returns HLS manifest URL for ExoPlayer. Returns 202 Accepted if transcoder not ready yet.
POST	/api/v1/streams/{id}/reset-viewers	Bearer (admin)	Reset viewerCount to 0 and batch-delete all viewer presence docs.

Analytics Endpoints

METHOD	PATH	AUTH	DESCRIPTION
GET	/api/v1/analytics/stream/{id}	None	Get analytics doc: currentViewers, peakViewers, totalJoins
GET	/api/v1/analytics/stream/{id}/viewers	None	Get full viewer history list ordered by join time

Admin API Endpoints (selected)

METHOD	PATH	AUTH	DESCRIPTION
GET	/api/v1/admin/dashboard	Admin	Summary stats: users, roles, live streams, total joins
GET	/api/v1/admin/users	Admin	List all users with role breakdown
PATCH	/api/v1/admin/users/{uid}/role	Admin	Promote/demote user role
GET	/api/v1/admin/streams	Admin	List all streams (including ended)
POST	/api/v1/admin/streams/{id}/end	Admin	Force-end a live stream
GET	/api/v1/admin/analytics/top	Admin	Top streams by peak viewers
GET	/api/v1/admin/analytics/sse	Admin	Server-Sent Events stream for live analytics updates
GET	/api/v1/admin/matches	Admin	List scheduled matches
POST	/api/v1/admin/notifications/send	Admin	Send push notification (stream_started / match_reminder)
GET	/api/v1/admin/health	Admin	Health status of all 6 services

5 Data Model

All data is stored in **Cloud Firestore** — a schemaless, real-time NoSQL document database. The platform uses the following top-level collections:

Collection: users

`uid` (string, PK)
`email` (string)
`displayName` (string)
`photoUrl` (string)
`favTeams` (string[])
`role` (viewer | broadcaster | admin)
`fcmToken` (string, optional)
`createdAt` , `updatedAt` (timestamp)

Collection: streams

`id` (string, e.g. "stream_1714000000000")
`title` (string)
`status` (live | ended)
`hlsUrl` (string — GCS path to m3u8)
`viewerCount` (int — managed by transactions)
`broadcasterUid` (string, FK → users)
`createdAt` , `updatedAt` (timestamp)

Subcollection: `viewers/{uid}` — presence docs (uid, joinedAt)

Collection: analytics

`streamId` (string, same as stream ID)
`currentViewers` (int64)
`peakViewers` (int64)
`totalJoins` (int64)
`updatedAt` (timestamp)

Subcollections:

`processedEvents/{eventId}` — idempotency log
`viewerHistory/{uid}` — per-user watch history (joinedAt, leftAt, joinCount, isWatching)

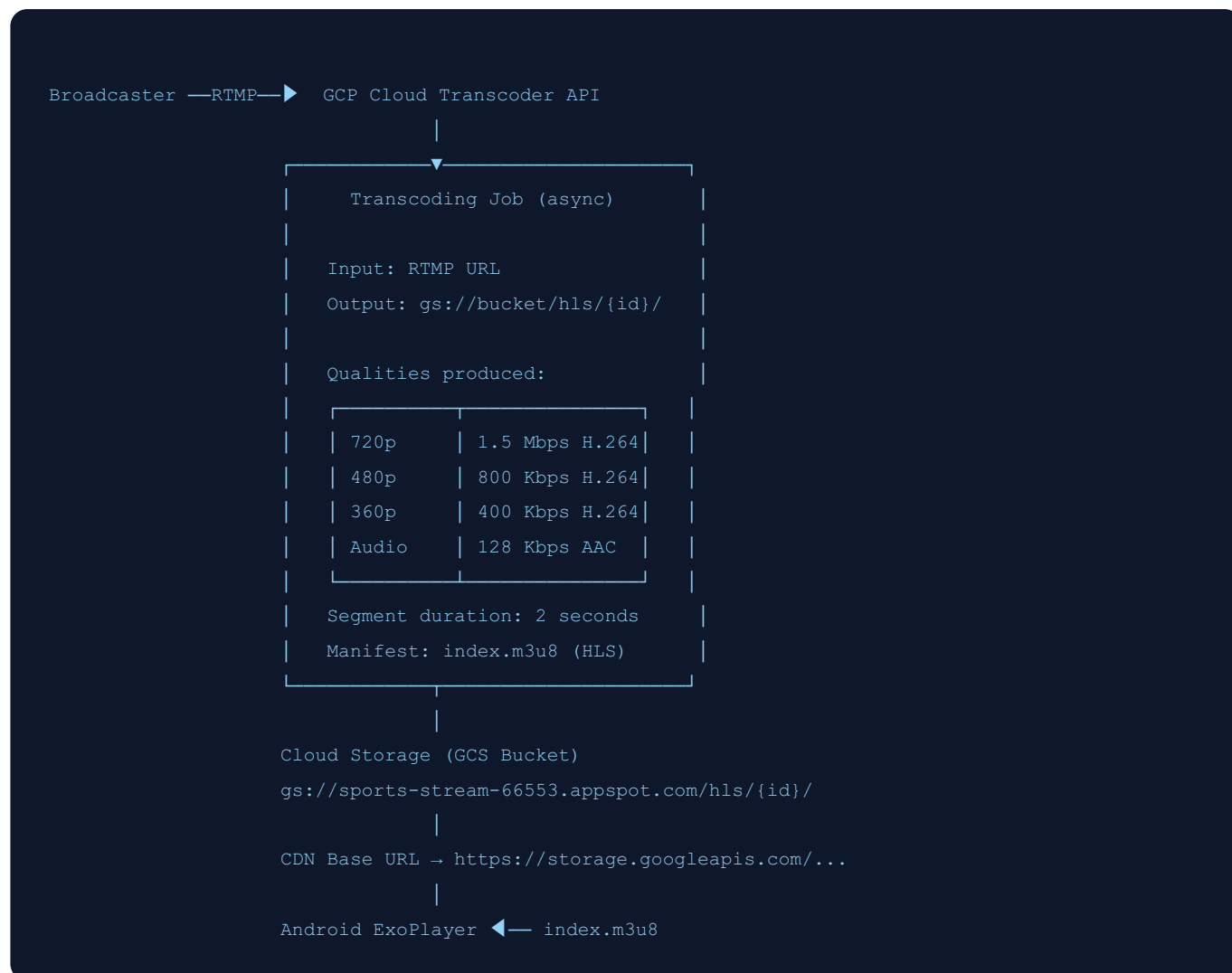
Collection: matches

`id` (string, e.g. "match_1714000000000")
`title` (string)
`scheduledAt` (timestamp)
`status` (scheduled | live | ended)
`createdAt` , `updatedAt` (timestamp)

Used for match schedule management in admin panel and reminder notifications.

6 Video Pipeline

The platform uses the **GCP Cloud Transcoder API** for fully managed video encoding — no self-managed FFmpeg servers required.



Flow: On stream creation, stream-service immediately writes the expected HLS URL to Firestore (so ExoPlayer can start retrying), then polls the transcoder job every 5 seconds for up to 10 minutes. When the job succeeds or fails, the stream is marked as `ended` and the viewer subcollection is batch-deleted.

7 Security & Authentication

Firestore JWT Authentication

The Android app authenticates with Google Sign-In and receives a Firebase ID token (JWT). Every protected API call includes this token as an

`Authorization: Bearer <token>` header.

Admin Panel Authentication

The admin web UI uses HMAC-SHA256 signed session cookies:

Token format: `base64(username|expiry|`

The `pkg/middleware` package verifies the token using Firebase Admin SDK and injects the verified UID into the request context. Invalid or expired tokens return HTTP 401.

`hmac_sig)`

- 12-hour session expiry
- HttpOnly + SameSite=Lax cookies
- Separate from Firebase auth

Admin API calls can also use Firebase tokens where the user's Firestore `role` is `admin`.

Role-Based Access Control: Roles are stored in Firestore and checked server-side on every request. A `viewer` cannot create streams. A `broadcaster` can only create streams but not access admin APIs. The `admin` role has full access to all endpoints. Roles can only be changed by admins via the admin panel.

Firestore Security Rules are also configured (`firestore.rules`) to enforce read/write access policies at the database level as a defense-in-depth measure.

8 Infrastructure & Deployment

Docker Build

A **multi-stage Dockerfile** builds all 6 Go binaries in a `golang:1.26-alpine` builder stage, then copies only the compiled binaries into a minimal `alpine:latest` runtime image with `ca-certificates` and `ffmpeg`.

A single `start.sh` script launches all 6 services in the background, with the gateway as the process that keeps the container alive. Only **port 8080** is exposed.

Deployment Options

Google Cloud Run (current)

Single container, single port (8080). Cloud Build CI/CD via `cloudbuild.yaml`. Fully managed, auto-scales to zero. Firebase credentials injected via environment variables. Suitable for production deployment with minimal ops overhead.

Kubernetes / K8s

Full Kubernetes deployment manifests in `k8s/deployments.yaml`. Each service as a separate Deployment. Horizontal Pod Autoscaler config in `k8s/hpa.yaml`. Secrets via Kubernetes Secrets (`sports-stream-secrets`). Suitable for multi-region or high-scale deployments.

Infrastructure as Code

Terraform configuration in `infra/terraform/` provisions GCP resources including Cloud Run services, Pub/Sub topics and subscriptions, Firestore database, and IAM bindings.

Environment Variables

VARIABLE	USED BY	DESCRIPTION
<code>GCP_PROJECT_ID</code>	All services	GCP project identifier
<code>FIREBASE_CREDENTIALS</code>	All services	Firebase service account JSON (can be file path or inline JSON)
<code>PORT</code>	All services	HTTP listen port (auto-set by Cloud Run)
<code>ADMIN_PANEL_USERNAME</code>	admin-service	Admin web UI username
<code>ADMIN_PANEL_PASSWORD</code>	admin-service	Admin web UI password
<code>ADMIN_PANEL_SESSION_SECRET</code>	admin-service	HMAC signing key for session cookies
<code>GCS_BUCKET</code>	stream-service	Cloud Storage bucket for HLS segments

<code>CDN_BASE_URL</code>	stream-service	Public CDN base URL for HLS manifest
<code>VIEWER_SUB</code>	analytics-service	Pub/Sub subscription name for viewer events
<code>STREAM_SUB</code>	analytics-service	Pub/Sub subscription name for stream events
<code>NOTIFICATION_SUB</code>	notification-service	Pub/Sub subscription name for notifications

9 Observability & Health

Health Endpoint (/health)

The gateway probes all 5 microservices' `/health` endpoints (2-second timeout each). Each service health check verifies Firestore connectivity with a `Limit(1)` query.

Returns: HTML dashboard (browser) or JSON (API client). HTTP 503 if any service is down.

Structured Logging

All services use `pkg/util.LogJSON()` for structured JSON log output. Each log line includes service name, level (info/error), event type, and contextual fields (requestId, uid, streamId).

Compatible with GCP Cloud Logging for centralized log aggregation and alerting.

Request Tracing

The gateway injects a unique `X-Request-Id` header on every request. If the client provides one, it is preserved. All service logs reference this request ID for distributed request tracing.

Real-time Analytics SSE

The admin panel connects to `/api/v1/admin/analytics/sse` which streams Firestore real-time snapshots via Server-Sent Events. Analytics data updates in the browser without any polling — changes appear instantly as Pub/Sub events are processed.

Load Testing: The project includes `k6-load-test.js` and `join_test.js` scripts for performance testing viewer join/leave concurrency and stream listing under load.

10 Future Roadmap — AWS Migration

The project documentation outlines a planned migration from GCP to AWS. The architecture mapping is as follows:

CURRENT (GCP)	TARGET (AWS)
Firebase Auth + JWT	Amazon Cognito
Cloud Firestore	Amazon DynamoDB
Cloud Pub/Sub	Amazon SNS + SQS
Firebase FCM	Amazon SNS Mobile Push
GCP Transcoder API	AWS Elemental MediaConvert
Cloud Storage (GCS)	Amazon S3 + CloudFront CDN
Cloud Run / GKE	Amazon ECS / EKS
Cloud Build	AWS CodePipeline / GitHub Actions
Cloud Logging	Amazon CloudWatch Metrics & Logs

Migration Strategy: The service-oriented architecture makes the migration incremental — each service can be migrated independently. The event-driven Pub/Sub pattern maps directly to SNS/SQS. The Firestore data model is document-oriented and can be translated to DynamoDB with appropriate partition key design.

Summary

Sports Stream Backend is a well-structured, production-ready live sports streaming platform that demonstrates modern backend engineering practices: microservice decomposition, event-driven architecture, idempotent operations, structured observability, and infrastructure-as-code. The system is designed to scale horizontally on both Cloud Run and Kubernetes, with clear paths for both vertical feature extension and horizontal cloud migration.

Key Engineering Highlights: Firestore transactions for race-condition-free viewer counting • SHA-256 event fingerprinting for exactly-once analytics processing • Multi-stage Docker build producing minimal Alpine images • HMAC-signed session cookies with no external session store • SSE-based real-time admin dashboard without WebSocket complexity • Single container deployment simplifying Cloud Run ops while maintaining service isolation